

Вот подробный конспект семинара по Модулям 7-8. Структура адаптирована под конвертацию в `.ipynb` : теория чередуется с кодовыми ячейками, каждое задание сопровождается примером решения.

Семинар: Модули 7-8. Безопасность и тестирование асинхронных приложений

Часть 1. Криптографические основы

1.1. Хеширование vs шифрование

Свойство	Шифрование	Хеширование
Обратимость	Да	Нет
Ключ	Требуется	Не требуется
Цель	Конфиденциальность	Целостность, проверка, хранение
Пример	AES-256-GCM	SHA-256, bcrypt

Пароли **хешируются**, а не шифруются. Если база скомпрометирована, злоумышленник не должен восстановить исходные пароли.

Почему не MD5/SHA-256 для паролей:

- SHA-256 на современном GPU обрабатывает миллиарды паролей в секунду.
- Пароли требуют **замедленных (deliberately slow)** хеш-функций — KDF (Key Derivation Functions).

Соль (Salt): случайная строка ≥ 16 байт, хранящаяся рядом с хешем. Без соли одинаковые пароли дают одинаковые хеши.

```
In [1]: Без соли:
Alice: password123 -> 482c811da5d5b4bc6d497ffa98491e38
Bob:   password123 -> 482c811da5d5b4bc6d497ffa98491e38  <- совпадение!

С солью:
```

```
Alice: password123 + salt_a -> a3f7...
Bob:   password123 + salt_b -> b9e2...  <- разные хеши
```

bcrypt: параметр cost factor (4-31, по умолчанию 12). Каждое увеличение cost удваивает время хеширования. Использует операции, неэффективные на GPU.

Argon2: победитель Password Hashing Competition (2015). Memory-hard: три параметра — memory, iterations, parallelism. Рекомендация OWASP (2023): Argon2id.

1.2. Хеширование паролей на практике

```
In [2]: from passlib.context import CryptContext

# bcrypt
pwd_context_bcrypt = CryptContext(schemes=["bcrypt"], deprecated="auto")
hashed_bcrypt = pwd_context_bcrypt.hash("mysecretpassword")
print(f"bcrypt hash: {hashed_bcrypt}")
is_valid = pwd_context_bcrypt.verify("mysecretpassword", hashed_bcrypt)
print(f"Verify correct: {is_valid}")
is_invalid = pwd_context_bcrypt.verify("wrongpassword", hashed_bcrypt)
print(f"Verify wrong: {is_invalid}")

# Argon2 (требуется: pip install argon2-cffi)
try:
    pwd_context_argon2 = CryptContext(schemes=["argon2"], deprecated="auto")
    hashed_argon2 = pwd_context_argon2.hash("mysecretpassword")
    print(f"Argon2 hash: {hashed_argon2[:50]}...")
except Exception as e:
    print(f"Argon2 not available: {e}")
```

1.3. JWT: структура и принципы

JWT (JSON Web Token, RFC 7519) — компактный способ передавать утверждения (claims) в виде подписанного JSON.

Структура: `header.payload.signature`, каждая часть в Base64Url.

Header:

```
In [3]: {"alg": "HS256", "typ": "JWT"}
```

Payload (Claims):

```
In [4]: {
    "sub": "1234567890",
    "name": "Alice",
    "iat": 1516239022,
    "exp": 1516242622,
    "scope": "predict:read predict:write"
}
```

Signature (HS256):

```
In [5]: HMAC_SHA256(Base64Url(header) + "." + Base64Url(payload), secret)
```

Stateful vs Stateless:

Подход	Stateful (Session ID)	Stateless (JWT)
Хранение	Сессия на сервере	Вся информация в токене
Проверка	Запрос к хранилищу	Проверка подписи локально
Отзыв	Мгновенный	Сложный (blacklist, короткий TTL)
Масштабируемость	Требует shared storage	Любой сервер проверяет сам
Размер	16-32 байта	Сотни байт - килобайты

Access Token + Refresh Token:

Токен	Срок жизни	Хранение	Назначение
Access Token	5-15 минут	Память приложения	Доступ к API
Refresh Token	7-30 дней	httpOnly cookie	Получение нового access token

1.4. Алгоритмы подписи JWT

HS256 (HMAC-SHA256): симметричный. Один secret для подписи и проверки. Простой, быстрый, но все сервисы знают секрет.

RS256 (RSA + SHA-256): асимметричный. Приватный ключ подписывает, публичный проверяет. Публичный ключ можно распространять безопасно. Медленнее, ключи большие

(2048+ бит).

ES256 (ECDSA + P-256): асимметричный на эллиптических кривых. Подпись в 2 раза короче RSA (64 байта vs 256 байт). Быстрее. Требует качественной генерации случайных чисел.

1.5. Реализация JWT в Python

```
In [6]: from datetime import datetime, timedelta, timezone
        from jose import JWTError, jwt

        SECRET_KEY = "your-secret-key-here-change-in-production"
        ALGORITHM = "HS256"
        ACCESS_TOKEN_EXPIRE_MINUTES = 15

        def create_access_token(data: dict, expires_delta: timedelta | None = None):
            to_encode = data.copy()
            if expires_delta:
                expire = datetime.now(timezone.utc) + expires_delta
            else:
                expire = datetime.now(timezone.utc) + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
            to_encode.update({"exp": expire, "iat": datetime.now(timezone.utc)})
            encoded_jwt = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
            return encoded_jwt

        def verify_token(token: str):
            try:
                payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
                return payload
            except JWTError:
                return None

        # Демонстрация
        token = create_access_token(data={"sub": "user123", "name": "Alice"})
        print(f"Token: {token[:50]}...")

        payload = verify_token(token)
        print(f"Payload: {payload}")

        # Просроченный токен
        expired_token = create_access_token(
            data={"sub": "user123"},
            expires_delta=timedelta(seconds=-1)
        )
```

```
expired_payload = verify_token(expired_token)
print(f"Expired payload: {expired_payload}") # None
```

Часть 2. Аутентификация и авторизация в FastAPI

2.1. OAuth2 Password Flow

```
In [7]: from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm

app = FastAPI()

# URL для получения токена
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

# "База данных" пользователей
fake_users_db = {
    "alice": {
        "username": "alice",
        "hashed_password": "$2b$12$...", # bcrypt hash of "secret"
        "role": "admin"
    },
    "bob": {
        "username": "bob",
        "hashed_password": "$2b$12$...",
        "role": "analyst"
    }
}

# Упрощенная проверка (в реальности – проверка bcrypt)
def verify_password(plain: str, hashed: str) -> bool:
    return plain == "secret" # заглушка для семинара

def authenticate_user(username: str, password: str):
    user = fake_users_db.get(username)
    if not user or not verify_password(password, user["hashed_password"]):
        return None
    return user

def create_access_token(data: dict):
    from jose import jwt
    from datetime import datetime, timezone, timedelta
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + timedelta(minutes=15)
```

```

to_encode.update({"exp": expire})
return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

# --- Endpoint для получения токена ---
@app.post("/token")
async def login(form_data: OAuth2PasswordRequestForm = Depends()):
    user = authenticate_user(form_data.username, form_data.password)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )
    access_token = create_access_token(data={"sub": user["username"]})
    return {"access_token": access_token, "token_type": "bearer"}

# --- Защищенный endpoint ---
def get_current_user(token: str = Depends(oauth2_scheme)):
    payload = verify_token(token)
    if payload is None:
        raise HTTPException(status_code=401, detail="Invalid token")
    username = payload.get("sub")
    if username not in fake_users_db:
        raise HTTPException(status_code=401, detail="User not found")
    return fake_users_db[username]

@app.get("/predictions")
async def read_predictions(user: dict = Depends(get_current_user)):
    return {"predictions": [], "user": user["username"]}

# uvicorn seminar_78:app --port 8000
# Получить токен: curl -X POST "http://localhost:8000/token" -d "username=alice&password=secret"
# Использовать: curl -H "Authorization: Bearer <token>" http://localhost:8000/predictions

```

2.2. RBAC: Role-Based Access Control

```

In [8]: from enum import Enum
        from fastapi import FastAPI, Depends, HTTPException, status

        app = FastAPI()

        class Role(str, Enum):

```

```

ADMIN = "admin"
DATA_SCIENTIST = "data_scientist"
ANALYST = "analyst"
API_CLIENT = "api_client"

ROLE_PERMISSIONS = {
    Role.ADMIN: {"user:*", "model:*", "prediction:*"},
    Role.DATA_SCIENTIST: {"model:read", "model:write", "prediction:read"},
    Role.ANALYST: {"prediction:read", "dashboard:read"},
    Role.API_CLIENT: {"prediction:create"},
}

# Расширенная "база данных"
fake_users_db = {
    "alice": {"username": "alice", "password": "secret", "role": Role.ADMIN},
    "bob": {"username": "bob", "password": "secret", "role": Role.ANALYST},
    "charlie": {"username": "charlie", "password": "secret", "role": Role.API_CLIENT},
}

class PermissionChecker:
    def __init__(self, required_permissions: list[str]):
        self.required_permissions = required_permissions

    def __call__(self, user: dict = Depends(get_current_user)):
        user_perms = ROLE_PERMISSIONS.get(user["role"], set())
        for required in self.required_permissions:
            if required in user_perms:
                continue
            resource, action = required.split(":")
            if f"{resource}:*" in user_perms:
                continue
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail=f"Permission {required} required"
            )
        return user

@app.post("/models/")
async def create_model(_: dict = Depends(PermissionChecker(["model:write"])))
    return {"message": "Model created"}

@app.get("/predictions/")
async def read_predictions(_: dict = Depends(PermissionChecker(["prediction:read"])))
    return {"predictions": [1, 2, 3]}

```

```

@app.post("/predict/")
async def create_prediction(_: dict = Depends(PermissionChecker(["prediction:create"])))
    return {"prediction": "result"}

# alice (admin) – может всё
# bob (analyst) – может читать predictions, но не создавать models
# charlie (api_client) – может только prediction:create

```

2.3. ABAC: Attribute-Based Access Control

```

In [9]: from datetime import datetime

# ABAC: проверка на основе атрибутов пользователя, ресурса, окружения
class Model:
    def __init__(self, id: int, owner_id: str, status: str):
        self.id = id
        self.owner_id = owner_id
        self.status = status # "draft" или "published"

def can_delete_model(user: dict, model: Model) -> bool:
    # Админ может всё
    if user.get("role") == Role.ADMIN:
        return True

    # Владелец может удалять свои черновики
    if model.owner_id == user.get("username") and model.status == "draft":
        return True

    # Ночью (00:00-06:00) никто не удаляет
    if datetime.now().hour < 6:
        return False
    return False

# Демонстрация
alice = {"username": "alice", "role": Role.ADMIN}
bob = {"username": "bob", "role": Role.ANALYST}

model_draft = Model(id=1, owner_id="bob", status="draft")
model_published = Model(id=2, owner_id="bob", status="published")

print(f"Alice deletes draft: {can_delete_model(alice, model_draft)}")      # True
(admin)

print(f"Bob deletes own draft: {can_delete_model(bob, model_draft)}")    # True
(owner + draft)

```



```
print(f"Bob deletes own published: {can_delete_model(bob, model_published)}") # False (published)
```

2.4. Защита приложения: CORS, CSRF, XSS

CORS (Cross-Origin Resource Sharing):

```
In [10]: from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://app.example.com"], # конкретные домены, не "*"
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE"],
    allow_headers=["*"],
)
```

Опасность: `allow_origins=["*"]` с `allow_credentials=True` — уязвимость. Любой сайт может делать аутентифицированные запросы.

CSRF (Cross-Site Request Forgery):

- Защита: `SameSite=strict` или `SameSite=lax` для cookies.
- Для API с JWT в заголовке CSRF неактуален (злоумышленник не может подделать заголовок с чужого домена).

XSS (Cross-Site Scripting):

- FastAPI + Pydantic по умолчанию экранируют JSON.
- Content-Security-Policy заголовок запрещает inline-скрипты.

SQL Injection:

- SQLAlchemy ORM защищает через параметризованные запросы.
- Raw SQL требует осторожности: используйте `text("... WHERE name = :name")` с параметрами.

2.5. Rate Limiting: Token Bucket

```

In [11]: import time
import asyncio

class TokenBucket:
    def __init__(self, capacity: int, rate: float):
        self.capacity = capacity # максимум токенов
        self.rate = rate # токенов в секунду
        self.tokens = capacity # текущее число токенов
        self.last_update = time.monotonic()

    async def consume(self, tokens: int = 1) -> bool:
        now = time.monotonic()
        elapsed = now - self.last_update
        self.tokens = min(self.capacity, self.tokens + elapsed * self.rate)
        self.last_update = now

        if self.tokens >= tokens:
            self.tokens -= tokens
            return True
        return False

# Демонстрация
async def demo_token_bucket():
    bucket = TokenBucket(capacity=5, rate=1) # 5 токенов burst, 1/сек восполнения

    # Первые 5 запросов — успешно (burst)
    for i in range(5):
        result = await bucket.consume()
        print(f"Request {i+1}: {'OK' if result else 'REJECTED'}")

    # 6-й запрос — отклонен (токенов нет)
    result = await bucket.consume()
    print(f"Request 6: {'OK' if result else 'REJECTED'}")

    # Ждем 2 секунды — накапливается 2 токена
    await asyncio.sleep(2)
    for i in range(3):
        result = await bucket.consume()
        print(f"Request {7+i}: {'OK' if result else 'REJECTED'}")

asyncio.run(demo_token_bucket())

```

2.6. Хранение секретов

Уровень	Подход	Пример
1	Жестко в коде	<code>SECRET = "abc123"</code> — категорически запрещено
2	Переменные окружения	<code>os.environ["SECRET_KEY"]</code> — минимум
3	Файлы секретов	Docker secrets, Kubernetes secrets
4	Vault	HashiCorp Vault, AWS Secrets Manager

```
In [12]: from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str
    secret_key: str
    algorithm: str = "HS256"
    access_token_expire_minutes: int = 15

    class Config:
        env_file = ".env"
        env_file_encoding = "utf-8"

# settings = Settings() # создается при импорте
```

Часть 3. Тестирование асинхронных приложений

3.1. Пирамида тестирования

Уровень	Скорость	Стоимость поддержки	Что ловит
Unit	Миллисекунды	Низкая	Ошибки логики, граничные условия
Integration	Секунды	Средняя	Ошибки взаимодействия компонентов
E2E	Минуты	Высокая	Ошибки конфигурации, инфраструктуры

Для микросервисов — бриллиант (test diamond):

- Много unit-тестов для внутренней логики.
- Еще больше интеграционных тестов (сервис + БД, кэш, брокер).
- Много контрактных тестов (consumer-driven contracts).
- Умеренное число E2E.

3.2. pytest-asyncio

```
In [13]: import pytest
import asyncio

# Базовый синтаксис
@pytest.mark.asyncio
async def test_async_add():
    async def async_add(a, b):
        await asyncio.sleep(0.001)
        return a + b

    result = await async_add(2, 3)
    assert result == 5

# Таймауты
@pytest.mark.asyncio
async def test_timeout_raises():
    async def slow():
        await asyncio.sleep(10)

    with pytest.raises(asyncio.TimeoutError):
        await asyncio.wait_for(slow(), timeout=0.1)

# Исключения в корутинах
@pytest.mark.asyncio
async def test_exception_in_task():
    async def failing():
        raise ValueError("boom")

    task = asyncio.create_task(failing())
    with pytest.raises(ValueError, match="boom"):
        await task

# async for
@pytest.mark.asyncio
async def test_async_generator():
    async def gen():
        for i in range(3):
```

```

        yield i

    result = [x async for x in gen()]
    assert result == [0, 1, 2]

# Arrange-Act-Assert
@pytest.mark.asyncio
async def test_user_service():
    from unittest.mock import AsyncMock

    # ARRANGE
    mock_repo = AsyncMock()
    mock_repo.create.return_value = {"id": 1, "name": "Alice"}

    # ACT
    result = await mock_repo.create(name="Alice")

    # ASSERT
    assert result["name"] == "Alice"
    mock_repo.create.assert_awaited_once_with(name="Alice")

# Запуск тестов
# pytest seminar_78.py -v

```

3.3. Моки: unittest.mock и pytest-mock

```

In [14]: from unittest.mock import Mock, AsyncMock, patch
import asyncio

# Синхронный мок
mock = Mock()
mock.return_value = 42
print(f"Mock result: {mock()}")

# Асинхронный мок
async_mock = AsyncMock()
async_mock.return_value = {"status": "ok"}
result = asyncio.run(async_mock())
print(f"AsyncMock result: {result}")

# Patch: патчим там, где используется
# Пример: myapp/services.py делает `from aiohttp import ClientSession`
# Патчим: mocker.patch("myapp.services.ClientSession.get")

```

```
# respx для httpx
try:
    import respx
    from httpx import Response
    print("respx available")
except ImportError:
    print("respx not installed (pip install respx)")
```

3.4. Интеграционные тесты: TestClient и AsyncClient

```
In [15]: from fastapi import FastAPI
from fastapi.testclient import TestClient

app = FastAPI()

@app.get("/")
async def read_root():
    return {"message": "Hello World"}

@app.get("/items/{item_id}")
async def read_item(item_id: int):
    return {"item_id": item_id}

# TestClient (синхронный)
client = TestClient(app)

def test_read_main():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "Hello World"}

def test_read_item():
    response = client.get("/items/42")
    assert response.status_code == 200
    assert response.json() == {"item_id": 42}

# AsyncClient (асинхронный)
try:
    import httpx
    import pytest

    @pytest.mark.asyncio
    async def test_async_client():
        async with httpx.AsyncClient(app=app, base_url="http://test") as client:
```

```

        response = await client.get("/")
        assert response.status_code == 200

# Конкурентные запросы
@pytest.mark.asyncio
async def test_concurrent_requests():
    async with httpx.AsyncClient(app=app, base_url="http://test") as client:
        responses = await asyncio.gather(*[client.get("/") for _ in range(1
0)])

        assert all(r.status_code == 200 for r in responses)

except ImportError:
    print("httpx not installed")

test_read_main()
test_read_item()
print("Sync tests passed")

```

3.5. Тестирование DI с dependency_overrides

```

In [16]: from fastapi import FastAPI, Depends
        from fastapi.testclient import TestClient

        app = FastAPI()

        async def get_db():
            return {"type": "real", "data": "postgres"}

        @app.get("/items/")
        async def read_items(db: dict = Depends(get_db)):
            return db

        # --- Тестирование ---
        class FakeDB:
            async def fetch(self, query):
                return [{"id": 1, "name": "Test"}]

        def override_get_db():
            return {"type": "fake", "data": "in-memory"}

        app.dependency_overrides[get_db] = override_get_db

        client = TestClient(app)

```

```
def test_with_fake_db():
    response = client.get("/items/")
    assert response.status_code == 200
    data = response.json()
    assert data["type"] == "fake"
    print(f"Test passed: {data}")

test_with_fake_db()

# Очистка
app.dependency_overrides.clear()
```

3.6. Фабрики данных: Faker

```
In [17]: try:
    from faker import Faker

    fake = Faker("ru_RU")
    print(f"Name: {fake.name()}")
    print(f"Email: {fake.email()}")
    print(f"Address: {fake.address()}")

    # Генерация списка пользователей
    users = [
        {"name": fake.name(), "email": fake.email(), "city": fake.city()}
        for _ in range(3)
    ]
    for u in users:
        print(u)

except ImportError:
    print("faker not installed (pip install faker)")
```

3.7. Нагрузочное тестирование: метрики

Latency: почему не среднее?

Распределение latency имеет тяжелый правый хвост. Среднее скрывает проблемы.

Персентиль	Смысл
p50 (median)	50% запросов быстрее. Типичный опыт.

Персентиль	Смысл
p95	95% запросов быстрее. "Худший нормальный случай".
p99	99% запросов быстрее. Выбросы.
p99.9	0.1% запросов медленнее. Критично для финансов.

Закон Литтла:

In [18]: $L = \lambda * W$

- L — среднее число запросов в системе.
- λ — интенсивность поступления (RPS).
- W — среднее время пребывания (latency).

Модель M/M/1:

In [19]: $\rho = \lambda * S$ (загрузка, S — время обслуживания)
 $Lq = \rho^2 / (1 - \rho)$ (среднее число запросов в очереди)
 $Wq = \rho * S / (1 - \rho)$ (среднее время ожидания)

При $\rho \rightarrow 1$ время ожидания стремится к бесконечности. Безопасный лимит: $\rho \approx 0.7-0.8$.

3.8. Профилирование: asyncio debug mode

```
In [20]: import asyncio

async def blocking_coroutine():
    # Имитация блокировки event loop
    import time
    time.sleep(0.5) # НЕ ДЕЛАЙТЕ ТАК! Используйте await asyncio.sleep()
    return "done"

async def good_coroutine():
    await asyncio.sleep(0.5)
    return "done"

# Debug mode покажет блокировку
async def demo_debug():
```

```
# Включить debug: asyncio.run(main(), debug=True)
# Или: PYTHONASYNCIODEBUG=1 uvicorn main:app
result = await blocking_coroutine()
print(result)

# asyncio.run(demo_debug(), debug=True)
```

Задания для самостоятельной работы

Задание 1. Хеширование и проверка пароля

Напишите функции `hash_password(password)` и `verify_password(plain, hashed)`, используя `passlib` с `bcrypt`. Продемонстрируйте, что одинаковые пароли дают разные хеши (из-за соли), и что проверка работает корректно.

```
In [21]: from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)

# Демонстрация
password = "mysecretpassword"

hash1 = hash_password(password)
hash2 = hash_password(password)

print(f"Hash 1: {hash1}")
print(f"Hash 2: {hash2}")
print(f"Different hashes: {hash1 != hash2}") # True (salt!)
print(f"Verify hash1: {verify_password(password, hash1)}") # True
print(f"Verify wrong: {verify_password('wrong', hash1)}") # False
```

Задание 2. Создание и проверка JWT

Напишите функции `create_jwt(payload, secret, expires_minutes)` и `decode_jwt(token, secret)`. Продемонстрируйте создание токена, успешную проверку и ошибку при истечении срока или неверном секрете.

```

In [22]: from datetime import datetime, timedelta, timezone
        from jose import JWTError, jwt

        SECRET = "my-super-secret-key"

        def create_jwt(payload: dict, secret: str = SECRET, expires_minutes: int = 15) -> str:
            to_encode = payload.copy()
            expire = datetime.now(timezone.utc) + timedelta(minutes=expires_minutes)
            to_encode.update({"exp": expire, "iat": datetime.now(timezone.utc)})
            return jwt.encode(to_encode, secret, algorithm="HS256")

        def decode_jwt(token: str, secret: str = SECRET) -> dict | None:
            try:
                return jwt.decode(token, secret, algorithms=["HS256"])
            except JWTError:
                return None

        # Успешный сценарий
        token = create_jwt({"sub": "user123", "name": "Alice"}, expires_minutes=60)
        print(f"Token: {token[:50]}...")

        payload = decode_jwt(token)
        print(f"Decoded: {payload}")

        # Истекший токен
        expired = create_jwt({"sub": "user123"}, expires_minutes=-1)
        print(f"Expired decode: {decode_jwt(expired)}") # None

        # Неверный секрет
        wrong_secret = decode_jwt(token, secret="wrong-secret")
        print(f"Wrong secret: {wrong_secret}") # None

```

Задание 3. FastAPI с OAuth2 Password Flow

Создайте FastAPI-приложение с endpoint'ами `/token` (логин) и `/me` (информация о текущем пользователе). Используйте `OAuth2PasswordBearer` и `OAuth2PasswordRequestForm`. "База данных" — словарь в памяти.

```

In [23]: from fastapi import FastAPI, Depends, HTTPException, status
        from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
        from jose import jwt
        from datetime import datetime, timezone, timedelta

```

```

app = FastAPI()
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
SECRET = "secret-key"

# База данных
users_db = {
    "alice": {"username": "alice", "password": "secret", "role": "admin"},
    "bob": {"username": "bob", "password": "secret", "role": "user"},
}

def create_token(username: str) -> str:
    expire = datetime.now(timezone.utc) + timedelta(minutes=30)
    return jwt.encode({"sub": username, "exp": expire}, SECRET, algorithm="HS256")

def get_current_user(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET, algorithms=["HS256"])
        username = payload.get("sub")
        if username not in users_db:
            raise HTTPException(status_code=401, detail="User not found")
        return users_db[username]
    except jwt.JWTError:
        raise HTTPException(status_code=401, detail="Invalid token")

@app.post("/token")
async def login(form_data: OAuth2PasswordRequestForm = Depends()):
    user = users_db.get(form_data.username)
    if not user or user["password"] != form_data.password:
        raise HTTPException(status_code=401, detail="Invalid credentials")
    return {"access_token": create_token(form_data.username), "token_type": "bearer"}

@app.get("/me")
async def read_me(user: dict = Depends(get_current_user)):
    return {"username": user["username"], "role": user["role"]}

# uvicorn seminar_78:app --port 8000
# curl -X POST "http://localhost:8000/token" -d "username=alice&password=secret"
# curl -H "Authorization: Bearer <token>" http://localhost:8000/me

```

Задание 4. RBAC с PermissionChecker

Расширьте задание 3: добавьте роли и `PermissionChecker`. Endpoint `/admin/` доступен только `admin`, `/predictions/` — `analyst` и выше, `/predict/` — `api_client` и выше.

```

In [24]: from enum import Enum
from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from jose import jwt
from datetime import datetime, timezone, timedelta

app = FastAPI()
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
SECRET = "secret-key"

class Role(Enum):
    ADMIN = "admin"
    ANALYST = "analyst"
    API_CLIENT = "api_client"

ROLE_PERMISSIONS = {
    Role.ADMIN: {"admin:*", "prediction:*", "predict:*"},
    Role.ANALYST: {"prediction:read", "predict:create"},
    Role.API_CLIENT: {"predict:create"},
}

users_db = {
    "alice": {"username": "alice", "password": "secret", "role": Role.ADMIN},
    "bob": {"username": "bob", "password": "secret", "role": Role.ANALYST},
    "charlie": {"username": "charlie", "password": "secret", "role": Role.API_CLI
ENT},
}

def create_token(username: str) -> str:
    expire = datetime.now(timezone.utc) + timedelta(minutes=30)
    return jwt.encode({"sub": username, "exp": expire}, SECRET, algorithm="HS25
6")

def get_current_user(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET, algorithms=["HS256"])
        return users_db[payload.get("sub")]
    except jwt.JWTError:
        raise HTTPException(status_code=401, detail="Invalid token")

class PermissionChecker:
    def __init__(self, required: list[str]):
        self.required = required

    def __call__(self, user: dict = Depends(get_current_user)):
        perms = ROLE_PERMISSIONS.get(user["role"], set())
        for req in self.required:

```

```

        if req in perms: continue
        r, a = req.split(":")
        if f"{r}:*" in perms: continue
        raise HTTPException(status_code=403, detail=f"Need {req}")
    return user

@app.post("/token")
async def login(form_data: OAuth2PasswordRequestForm = Depends()):
    user = users_db.get(form_data.username)
    if not user or user["password"] != form_data.password:
        raise HTTPException(status_code=401, detail="Invalid credentials")
    return {"access_token": create_token(form_data.username), "token_type": "bearer"}

@app.get("/admin/")
async def admin_only(_: dict = Depends(PermissionChecker(["admin:*"])))
    return {"message": "Admin panel"}

@app.get("/predictions/")
async def read_predictions(_: dict = Depends(PermissionChecker(["prediction:read"])))
    return {"predictions": [1, 2, 3]}

@app.post("/predict/")
async def create_predict(_: dict = Depends(PermissionChecker(["predict:create"])))
    return {"prediction": "done"}

# alice -> все endpoint'ы
# bob -> /predictions/ и /predict/
# charlie -> только /predict/

```

Задание 5. Token Bucket Rate Limiter

Реализуйте класс `TokenBucket` и интегрируйте его в FastAPI как зависимость. Endpoint `/api/` должен отклонять запросы, если токенов нет (HTTP 429).

```

In [25]: from fastapi import FastAPI, Depends, HTTPException
import time

app = FastAPI()

class TokenBucket:
    def __init__(self, capacity: int, rate: float):

```

```

    self.capacity = capacity
    self.rate = rate
    self.tokens = float(capacity)
    self.last_update = time.monotonic()

    def consume(self, tokens: int = 1) -> bool:
        now = time.monotonic()
        elapsed = now - self.last_update
        self.tokens = min(self.capacity, self.tokens + elapsed * self.rate)
        self.last_update = now

        if self.tokens >= tokens:
            self.tokens -= tokens
            return True
        return False

# Глобальный bucket: 3 токена burst, 1/сек восполнение
bucket = TokenBucket(capacity=3, rate=1)

def rate_limit():
    if not bucket.consume():
        raise HTTPException(status_code=429, detail="Too many requests")

@app.get("/api/")
async def api_endpoint(_: None = Depends(rate_limit)):
    return {"message": "Success"}

# uvicorn seminar_78:app --port 8000
# for i in {1..5}; do curl http://localhost:8000/api/; echo; done
# Первые 3 -> 200, остальные -> 429
# Подождать 2 сек -> снова 2 токена

```

Задание 6. Unit-тест с pytest-asyncio

Напишите тесты для асинхронной функции `divide(a, b)`, которая возвращает `a / b` или выбрасывает `ValueError` при `b == 0`. Используйте `pytest.mark.asyncio`.

```

In [26]: import pytest
import asyncio

async def divide(a: float, b: float) -> float:
    await asyncio.sleep(0.001) # имитация асинхронной работы
    if b == 0:
        raise ValueError("Cannot divide by zero")

```

```

        return a / b

@pytest.mark.asyncio
async def test_divide_normal():
    result = await divide(10, 2)
    assert result == 5.0

@pytest.mark.asyncio
async def test_divide_negative():
    result = await divide(-10, 2)
    assert result == -5.0

@pytest.mark.asyncio
async def test_divide_by_zero():
    with pytest.raises(ValueError, match="Cannot divide by zero"):
        await divide(10, 0)

@pytest.mark.asyncio
async def test_divide_float():
    result = await divide(7, 3)
    assert abs(result - 2.333) < 0.001

# pytest seminar_78.py::test_divide_normal -v

```

Задание 7. AsyncMock для асинхронных зависимостей

Напишите тест для сервиса `PredictionService`, который зависит от `MLModel.predict()`. Замокайте `predict` через `AsyncMock` и проверьте, что он был вызван с правильными аргументами.

```

In [27]: import pytest
        from unittest.mock import AsyncMock

        class MLModel:
            async def predict(self, features: list[float]) -> list[float]:
                raise NotImplementedError

        class PredictionService:
            def __init__(self, model: MLModel):
                self._model = model

            async def predict(self, features: list[float]) -> dict:
                result = await self._model.predict(features)
                return {"input": features, "output": result, "status": "ok"}

```



```

@pytest.mark.asyncio
async def test_prediction_service():
    # ARRANGE
    mock_model = AsyncMock(spec=MLModel)
    mock_model.predict.return_value = [0.1, 0.9]

    service = PredictionService(mock_model)

    # ACT
    result = await service.predict([1.0, 2.0, 3.0])

    # ASSERT
    assert result["status"] == "ok"
    assert result["output"] == [0.1, 0.9]
    mock_model.predict.assert_awaited_once_with([1.0, 2.0, 3.0])

# pytest seminar_78.py::test_prediction_service -v

```

Задание 8. Интеграционный тест с TestClient

Создайте FastAPI-приложение с endpoint'ами `POST /users/` (создание) и `GET /users/{id}` (получение). Напишите интеграционные тесты через `TestClient`.

```

In [28]: from fastapi import FastAPI
from fastapi.testclient import TestClient
from pydantic import BaseModel

app = FastAPI()

class UserCreate(BaseModel):
    name: str
    email: str

class User(BaseModel):
    id: int
    name: str
    email: str

# In-memory "база данных"
_users_db = {}
_counter = 1

@app.post("/users/")

```

```

async def create_user(user: UserCreate):
    global _counter
    new_user = User(id=_counter, name=user.name, email=user.email)
    _users_db[_counter] = new_user
    _counter += 1
    return new_user

@app.get("/users/{user_id}")
async def get_user(user_id: int):
    user = _users_db.get(user_id)
    if not user:
        return {"error": "Not found"}
    return user

# --- Тесты ---
client = TestClient(app)

def test_create_user():
    response = client.post("/users/", json={"name": "Alice", "email": "a@b.com"})
    assert response.status_code == 200
    data = response.json()
    assert data["name"] == "Alice"
    assert data["id"] == 1

def test_get_user():
    response = client.get("/users/1")
    assert response.status_code == 200
    data = response.json()
    assert data["name"] == "Alice"

def test_get_nonexistent_user():
    response = client.get("/users/999")
    assert response.status_code == 200
    assert response.json()["error"] == "Not found"

def test_create_multiple_users():
    response = client.post("/users/", json={"name": "Bob", "email": "b@c.com"})
    assert response.json()["id"] == 2

test_create_user()
test_get_user()
test_get_nonexistent_user()
test_create_multiple_users()
print("All integration tests passed!")

```

Задание 9. Тестирование с `dependency_overrides`

Напишите приложение с зависимостью `get_config`, возвращающей конфигурацию. В тестах переопределите её на фейковую. Убедитесь, что `dependency_overrides.clear()` вызывается после тестов.

```
In [29]: from fastapi import FastAPI, Depends
from fastapi.testclient import TestClient

app = FastAPI()

class Config:
    def __init__(self):
        self.db_url = "postgresql://real"
        self.debug = False

def get_config():
    return Config()

@app.get("/config/")
async def read_config(config: Config = Depends(get_config)):
    return {"db_url": config.db_url, "debug": config.debug}

# --- Тесты ---
class FakeConfig:
    db_url = "sqlite:///memory:"
    debug = True

def test_config_override():
    app.dependency_overrides[get_config] = lambda: FakeConfig()

    client = TestClient(app)
    response = client.get("/config/")
    assert response.status_code == 200
    data = response.json()
    assert data["db_url"] == "sqlite:///memory:"
    assert data["debug"] is True
    print(f"Test passed: {data}")

# Очистка!
app.dependency_overrides.clear()

# Проверяем, что очистка работает
response = client.get("/config/")
data = response.json()
```

```
assert data["db_url"] == "postgresql://real"
print(f"After clear: {data}")

test_config_override()
```

Задание 10. Тестирование таймаутов

Напишите тест для функции `fetch_with_timeout(url, timeout)`, которая делает HTTP-запрос с таймаутом. Замокните HTTP-клиент и проверьте, что при превышении таймаута выбрасывается `asyncio.TimeoutError`.

```
In [30]: import pytest
import asyncio
from unittest.mock import AsyncMock, patch

async def fetch_with_timeout(url: str, timeout: float):
    # Имитация HTTP-запроса
    await asyncio.sleep(1) # "медленный" сервер
    return {"data": "response"}

@pytest.mark.asyncio
async def test_timeout_success():
    with patch("seminar_78.fetch_with_timeout", new_callable=AsyncMock) as mock:
        mock.return_value = {"data": "response"}
        result = await fetch_with_timeout("http://api.com", timeout=2.0)
        # Примечание: здесь мы тестируем патченную версию
        # В реальности тестируем wrapper с asyncio.wait_for

@pytest.mark.asyncio
async def test_timeout_raises():
    async def slow_fetch(url: str, timeout: float):
        await asyncio.sleep(10) # очень медленно
        return {"data": "response"}

    with pytest.raises(asyncio.TimeoutError):
        await asyncio.wait_for(slow_fetch("http://api.com", 2.0), timeout=0.1)

# pytest seminar_78.py::test_timeout_raises -v
```

Задание 11. Генерация фейковых данных с Faker

Напишите функцию `generate_users(n)`, которая генерирует `n` фейковых пользователей с именем, email, адресом и возрастом. Используйте `faker`.

```

In [31]: try:
        from faker import Faker

        fake = Faker()

        def generate_users(n: int) -> list[dict]:
            return [
                {
                    "name": fake.name(),
                    "email": fake.email(),
                    "address": fake.address(),
                    "age": fake.random_int(min=18, max=90),
                }
                for _ in range(n)
            ]

        users = generate_users(5)
        for u in users:
            print(u)

    except ImportError:
        print("faker not installed (pip install faker)")

        # Fallback без faker
        def generate_users(n: int) -> list[dict]:
            return [
                {
                    "name": f"User {i}",
                    "email": f"user{i}@example.com",
                    "address": f"Address {i}",
                    "age": 20 + i,
                }
                for i in range(n)
            ]

        users = generate_users(3)
        for u in users:
            print(u)

```

Задание 12. Полноценный тестовый сценарий: регистрация + логин + доступ

Напишите полный цикл: регистрация пользователя, логин, получение токена, доступ к защищенному ресурсу. Используйте `TestClient` и in-memory хранилище.

```
In [32]: from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from fastapi.testclient import TestClient
from pydantic import BaseModel, EmailStr
from jose import jwt
from datetime import datetime, timezone, timedelta
from passlib.context import CryptContext

app = FastAPI()
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")
SECRET = "test-secret-key"
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

# --- Модели ---
class UserRegister(BaseModel):
    username: str
    email: EmailStr
    password: str

class UserInDB(BaseModel):
    username: str
    email: str
    hashed_password: str
    is_active: bool = False

# --- Хранилище ---
_users_db: dict[str, UserInDB] = {}

# --- Утилиты ---
def hash_pwd(p: str) -> str:
    return pwd_context.hash(p)

def verify_pwd(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)

def create_token(username: str, token_type: str = "access") -> str:
    expire = datetime.now(timezone.utc) + timedelta(minutes=30)
    return jwt.encode({"sub": username, "type": token_type, "exp": expire}, SECRET, algorithm="HS256")

def get_current_user(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET, algorithms=["HS256"])
        username = payload.get("sub")
        if username not in _users_db:
            raise HTTPException(status_code=401, detail="User not found")
```

```

        return _users_db[username]
    except jwt.JWTError:
        raise HTTPException(status_code=401, detail="Invalid token")

# --- Endpoint'ы ---
@app.post("/register/")
async def register(data: UserRegister):
    if data.username in _users_db:
        raise HTTPException(status_code=400, detail="Username exists")
    user = UserInDB(
        username=data.username,
        email=data.email,
        hashed_password=hash_pwd(data.password)
    )
    _users_db[data.username] = user
    return {"message": "Registered successfully"}

@app.post("/token")
async def login(form_data: OAuth2PasswordRequestForm = Depends()):
    user = _users_db.get(form_data.username)
    if not user or not verify_pwd(form_data.password, user.hashed_password):
        raise HTTPException(status_code=401, detail="Invalid credentials")
    return {"access_token": create_token(form_data.username), "token_type": "bearer"}

@app.get("/profile/")
async def profile(user: UserInDB = Depends(get_current_user)):
    return {"username": user.username, "email": user.email, "active": user.is_active}

# --- Интеграционный тест ---
def test_full_flow():
    client = TestClient(app)

    # 1. Регистрация
    response = client.post("/register/", json={
        "username": "alice",
        "email": "alice@example.com",
        "password": "secret123"
    })
    assert response.status_code == 200
    print(f"Register: {response.json()}")

    # 2. Логин
    response = client.post("/token", data={
        "username": "alice",

```

```

        "password": "secret123"
    })
    assert response.status_code == 200
    token = response.json()["access_token"]
    print(f"Login: got token")

# 3. Доступ к профилю
response = client.get("/profile/", headers={"Authorization": f"Bearer {token}"})
assert response.status_code == 200
data = response.json()
assert data["username"] == "alice"
print(f"Profile: {data}")

# 4. Неверный пароль
response = client.post("/token", data={
    "username": "alice",
    "password": "wrong"
})
assert response.status_code == 401
print(f"Wrong password: {response.status_code}")

# 5. Без токена
response = client.get("/profile/")
assert response.status_code == 401
print(f"No token: {response.status_code}")

print("\nAll tests passed!")

test_full_flow()

```

Итоговая сводка

Концепция	Модуль	Ключевой API / Паттерн
Хеширование паролей	7	bcrypt, Argon2id через passlib
Соль	7	Случайная строка >= 16 байт
JWT	7	<code>header.payload.signature</code> , Base64Url
HS256/RS256/ES256	7	Симметричный vs асимметричный
Access + Refresh Token	7	Короткий access, длинный refresh в httpOnly

Концепция	Модуль	Ключевой API / Паттерн
OAuth2 Password Flow	7	<code>OAuth2PasswordBearer</code> , <code>OAuth2PasswordRequestForm</code>
RBAC	7	Роли -> разрешения -> ресурсы
ABAC	7	Атрибуты пользователя, ресурса, окружения
CORS	7	<code>CORSMiddleware</code> , <code>allow_origins</code>
CSRF/XSS	7	SameSite, CSP, экранирование
Rate Limiting	7	Token Bucket, Sliding Window
TLS	7	Let's Encrypt, терминация SSL
Хранение секретов	7	Vault, env vars, никогда в коде
Пирамида тестирования	8	Unit -> Integration -> E2E
pytest-asyncio	8	<code>@pytest.mark.asyncio</code> , режим <code>strict</code>
AsyncMock	8	<code>assert_awaited_once_with</code>
respx	8	Перехват HTTPX-запросов
TestClient	8	Синхронный клиент для ASGI
AsyncClient	8	Конкурентные тесты, streaming
dependency_overrides	8	Подмена зависимостей + очистка
Faker	8	Генерация тестовых данных
p99 latency	8	Персентили вместо среднего
Закон Литтла	8	$L = \lambda * W$
M/M/1 модель	8	$\rho = \lambda * S$, точка насыщения
py-spy / memray	8	Профилировщики CPU и памяти
asyncio debug	8	<code>PYTHONASYNCIODEBUG=1</code>

Чек-лист для самопроверки

- ☐ Я знаю, почему SHA-256 нельзя использовать для паролей.
- ☐ Я могу объяснить разницу между хешированием и шифрованием.
- ☐ Я понимаю структуру JWT и назначение каждой части.
- ☐ Я знаю разницу между HS256, RS256 и ES256.
- ☐ Я могу реализовать OAuth2 Password Flow в FastAPI.
- ☐ Я понимаю разницу между аутентификацией и авторизацией.
- ☐ Я могу написать RBAC с PermissionChecker.
- ☐ Я знаю, как защитить приложение от CORS, CSRF, XSS.
- ☐ Я понимаю принцип Token Bucket для rate limiting.
- ☐ Я умею писать unit-тесты для асинхронных функций.
- ☐ Я знаю разницу между Mock и AsyncMock.
- ☐ Я могу писать интеграционные тесты с TestClient.
- ☐ Я понимаю, зачем очищать `dependency_overrides`.
- ☐ Я знаю, почему для latency используют персентили, а не среднее.
- ☐ Я понимаю закон Литтла и его применение к capacity planning.